

Neyron tarmoqlarga asoslangan so'z embeddinglari: “Word2Vec” (CBOW).

Tabiiy tilni qayta ishlash — 6-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №6

24-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: so'zlarni o'rganilgan vektorlar bilan ifodalaymiz
- 2 Nega o'rganiladigan zich vektorlar?
- 3 CBOW arxitekturasini kontekstdan markazga
- 4 Neyron tarmoqni o'qitish: yo'qotish va orqaga tarqatish
- 5 O'rganilgan embeddinglarni baholash
- 6 Xulosa va 6-amaliyotga ko'rsatmalar

Takrorlash: o'tgan darsdan ikki savol

1-savol

5-ma'ruzada N-gram modeli so'zlarni diskret (alohida) tokenlar deb hisobladi: `kitob` va `kitobni` — butunlay bog'liqsiz. Bu agglutinativ o'zbek tilida qanday muammoga olib keladi?

2-savol

5-amaliyotda avtomatik to'ldirish sanashga (*count*) tayandi. Sanash ikki so'zning **ma'no o'xshashligini** o'rgata oladimi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

5-ma'ruzada N-gram modeli so'zlarni diskret (alohida) tokenlar deb hisobladi: *kitob* va *kitobni* — butunlay bog'liqsiz. Bu agglutinativ o'zbek tilida qanday muammoga olib keladi?

2-savol

5-amaliyotda avtomatik to'ldirish sanashga (*count*) tayandi. Sanash ikki so'zning **ma'no o'xshashligini** o'rgata oladimi?

1-javob

Lug'at portlaydi (*vocabulary explosion*): har qo'shimchali shakl alohida token. Lug'at katalashadi, sanoqlar siyraklashadi.

2-javob

Yo'q. Sanash faqat “birga uchradimi” ni biladi; *film* va *kino* ni yaqin deb bilmaydi. Bugun shuni o'rganamiz.

Bugungi dars maqsadlari

- ① One-hot (diskret) ifoda nega yetarli emasligini — siyraklik va o'xshashlikni ko'rsata olmaslikni — **keltirib chiqara olasiz**.
- ② CBOW (*Continuous Bag of Words*) arxitekturasini — kontekst vektorlarini o'rtachalab markaziy so'zni bashorat qilishni — **tushuntira olasiz** ($h = [0.3, 0.5]$).
- ③ Softmax bashorati, o'zaro entropiya (*cross-entropy*) yo'qotishi va gradientning bir qadamini qo'lda **hisoblay olasiz**.
- ④ O'rganilgan embeddinglar sifatini kosinus o'xshashligi va analogiya orqali **baholay olasiz**.

Oldingi bilimlar: 3-ma'ruzadan vektor va kosinus o'xshashligi; softmax g'oyasi bugun kiritiladi. Matematikadan: vektor o'rtachasi, ko'paytma, exp, ln va hosila (zanjir qoidasi).

Tsikl	Mavzu	Natija
1	Nega zich vektor? (one-hot $\rightarrow$ embedding)	$\cos(\text{one-hot}) = 0$
2	CBOW arxitekturaları (kontekst $\rightarrow$ markaz)	$h = [0.3, 0.5]$
3	O'qitish: softmax, yo'qotish, gradient	$\hat{y}(\text{nlp}) = 0.690$
4	Embeddinglarnı baholash (kosinus, analogiya)	$\cos(\text{film}, \text{kino}) = 0.96$
–	Xulosa va 6-amaliyotga ko'prik	6-amaliyot yo'nalishi

Bu dars 2-haftani davom ettiradi. 5-ma'ruzađagi ehtimollik modellaridan zich (o'rganilgan) ifodalarga o'tamiz.

Bugungi muammo: film va kino yaqin ekanini qanday bilamiz?

Sodda yechim qayerda qiynaladi?

1. One-hot: har so'z alohida o'q

film = (1, 0, 0, ...), kino = (0, 1, 0, ...).

Ular **ortogonal**: o'xshashlik 0. Lekin ma'no jihatdan yaqin.

2. Tayyor model qamrovi past bo'lishi mumkin

Tayyor GloVe/Word2Vec modellarida ko'p o'zbekcha so'zlar OOV bo'ladi.

OOV (*out-of-vocabulary*) — model lug'atida yo'q, ya'ni model tanimaydigan so'z.

Bugungi yechim

- 1 So'z uchun **zich** (*dense*), kichik o'lchamli vektor quramiz.
- 2 Vektorni tayyor qoida bilan emas, **korpusdan** o'rganamiz.
- 3 O'xshash kontekstda kelgan so'zlar fazoda yaqinlashadi:

$$\text{film} \approx \text{kino}.$$

Bu yondashuvning klassik nomi — **Word2Vec**.
Bugun uning **CBOW** (*Continuous Bag of Words*) variantini o'rganamiz.

3-ma'ruza → 6-ma'ruza: embeddingdan FOYDALANISH va uni O'QITISH

Tayyor (*pre-trained*) embeddinglarni oldik va ishlatdik:

- kosinus o'xshashligi, `most_similar`;
- so'z analogiyalari; PCA vizualizatsiya.

Ochiq savol: **bu vektorlar qayerdan keladi?**

6-ma'ruza — O'QITISH

Bugun shu embeddinglarni **korpusdan o'rganamiz**:

- **Word2Vec (CBOW)** kontekstdan markaziy so'zni bashorat qiladi;
- shu jarayonda **embedding matritsa W** shakllanadi.

Natija — aynan 3-ma'ruzada ishlatgan turdagi vektorlar.

Ya'ni: 3-ma'ruza — embeddingni **ishlatish**; 6-ma'ruza — embeddingni **yaratish (o'qitish)**.

Prezentatsiya rejasi

- 1 Kirish: so'zlarni o'rganilgan vektorlar bilan ifodalaymiz
- 2 Nega o'rganiladigan zich vektorlar?
- 3 CBOW arxitekturasini kontekstdan markazga
- 4 Neyron tarmoqni o'qitish: yo'qotish va orqaga tarqatish
- 5 O'rganilgan embeddinglarni baholash
- 6 Xulosa va 6-amaliyotga ko'rsatmalar

Intuitsiya: so'z ma'nosi uning kontekstida ko'rinadi

“So'zni uning yonida keladigan so'zlar orqali taniymiz.”

film va kino ko'pincha o'xshash kontekstlarda uchraydi:

- ... yaxshi ko'rdim
- ... qiziq chiqdi
- ... tomosha qildim

Demak, vektorlar **kontekstdan o'rgatilsa**, film va kino fazoda yaqin joylashishi kerak.

One-hot ifodada har so'z alohida o'q: film va kino yaqin chiqmaydi. Zich vektor fazosida esa yaqin ma'no $\Rightarrow$ yaqin nuqta.



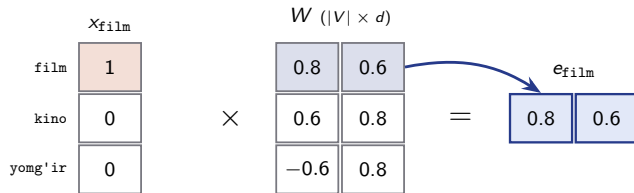
O'xshash kontekstlar takrorlansa, model bu ikki so'zni semantik jihatdan yaqin deb o'rganadi.

One-hot va embedding (zich vektor): rasmiy ta'rif

$$x_w \in \{0, 1\}^{|V|}, \quad \sum_i x_{w,i} = 1, \quad e_w = W^T x_w \in \mathbb{R}^d$$

Belgilashlar

- $|V|$ — lug'at hajmi;
- x_w — w so'zining one-hot vektori;
- W — embedding matritsasi;
- d — embedding o'lchami;
- e_w — w so'zining zich vektori.



Intuitsiya: one-hot vektordagi "1" qaysi so'z tanlanganini ko'rsatadi. Ko'paytirish amalda W dan shu so'zga tegishli bitta qatorni olishga teng.

(soddalashtirilgan o'quv misoli)

$d \ll |V|$: odatda $|V| \sim 10^5$, lekin $d \sim 100-300$. One-hot siyrak; embedding esa zich va o'rganilgan.

Keltirib chiqarish: nega one-hot semantik yaqinlikni ko'rsata olmaydi?

1-qadam. Har xil so'zlarning one-hot vektorlari turli pozitsiyada 1 ga ega.

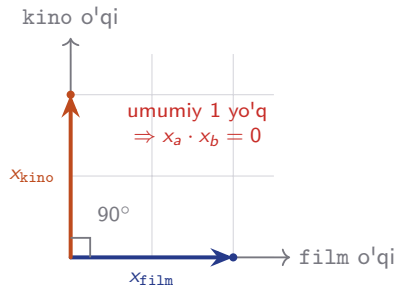
2-qadam. Skalyar ko'paytmada umumiy 1 yo'q:

$$x_a \cdot x_b = \sum_i x_{a,i} x_{b,i} = 0 \quad (a \neq b)$$

3-qadam. Shuning uchun kosinus ham nol:

$$\cos(x_a, x_b) = \frac{x_a \cdot x_b}{\|x_a\| \|x_b\|} = \frac{0}{1 \cdot 1} = 0.$$

One-hot fazoda **har xil so'zlar bir-biriga doim 0 o'xshash**. Shuning uchun film va kino semantik yaqinligi ko'rinmaydi.



Intuitsiya: har so'z alohida o'qda turadi. Burchak 90° , demak $\cos = 0$.

Qo'lda misol: one-hotda har xil so'zlar o'xshashligi nol

film = (1, 0, 0), kino = (0, 1, 0), yomg'ir = (0, 0, 1).

Hisoblaymiz:

$$x_{\text{film}} \cdot x_{\text{kino}} = 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 = 0$$

$$\cos(\text{film}, \text{kino}) = \frac{x_{\text{film}} \cdot x_{\text{kino}}}{\|x_{\text{film}}\| \|x_{\text{kino}}\|} = \frac{0}{1 \cdot 1} = 0$$

film va kino ma'noda yaqin, lekin one-hot ularni film va yomg'ir kabi bir xil darajada bog'liqsiz deb ko'rsatadi: har ikkisi bilan ham kosinus 0.

film	kino	yomg'ir
1	0	0
0	1	0
0	0	1

har xil juft uchun
 $x_a \cdot x_b = 0$

Intuitsiya: har bir so'z alohida o'qda turadi. Shuning uchun ular bir-biriga "tegmaydi".

Sizning vazifangiz: zich vektorlarda o'xshashlik

Vazifa

O'rganilgan zich vektorlar:

$$e_{\text{film}} = [0.8, 0.6], \quad e_{\text{kino}} = [0.6, 0.8].$$

Har ikkisi birlik vektor:

$$\|e_{\text{film}}\| = \|e_{\text{kino}}\| = 1.$$

$$\cos(\text{film}, \text{kino}) = ?$$

Sizning vazifangiz: zich vektorlarda o'xshashlik

Vazifa

$$e_{\text{film}} = [0.8, 0.6], \quad e_{\text{kino}} = [0.6, 0.8].$$

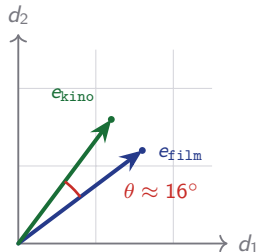
$$\cos(\text{film}, \text{kino}) = ?$$

Javob

$$e_{\text{film}} \cdot e_{\text{kino}} = 0.8 \cdot 0.6 + 0.6 \cdot 0.8 = 0.96$$

$$\cos = \frac{0.96}{1 \cdot 1} = 0.96$$

One-hotda 0 edi; zich fazoda 0.96 — yaqinlik ko'rindi.



Intuitsiya: yaqin ma'no $\Rightarrow$ vektorlar deyarli bir yo'nalishda.

Kod va formula: one-hotdan embedding qatorini olish

Python kodi

```
import numpy as np

V = ["film", "kino", "yomg'ir"]

W = np.array([[ 0.8, 0.6],      # film
               [ 0.6, 0.8],      # kino
               [-0.6, 0.8]])      # yomg'ir

def onehot(w):
    x = np.zeros(len(V))
    x[V.index(w)] = 1
    return x

x = onehot("film")
e_film = W.T @ x                # [0.8, 0.6]

# Amalda tezroq usul:
e_film_fast = W[V.index("film")]
```

Kod va formula mosligi

Kod	Formula
onehot(w)	x_w
W	$W \in \mathbb{R}^{ V \times d}$
W.T @ x	$W^T x_w = e_w$
W[index]	kerakli qator

One-hotdagi 1 qaysi so'z tanlanganini bildiradi. Shuning uchun $W^T x_w$ amalda W dan kerakli qatorni olishga teng.

$$e_{\text{film}} = [0.8, 0.6]$$

Tuzoq: embedding o'zicha paydo bo'lmaydi — uni o'rgatish kerak

Tasodifiy boshlang'ich vektor ma'nosiz

W ni tasodifiy qiymatlar bilan boshlaymiz, masalan:

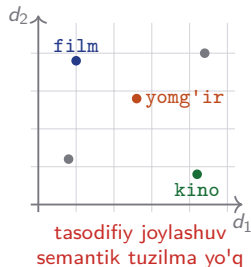
$$W_{ij} \sim \mathcal{N}(0, 0.01).$$

Bu paytda film, kino, yomg'ir vektorlari hali hech qanday semantik ma'no anglatmaydi.

Zich vektorning foydasi o'z-o'zidan emas, **o'qitish jarayonidan** keladi.

Savol: W ni qanday yangilaymizki, o'xshash kontekstda kelgan so'zlar bir-biriga yaqinlashsin?

Keyingi bosqichlarda: **CBOW** va *backpropagation* (orqaga tarqatish).



O'qitishdan oldin yaqinlik tasodifiy. O'qitishdan keyin kontekstdosh so'zlar yaqinlashadi.

Prezentatsiya rejasi

- 1 Kirish: so'zlarni o'rganilgan vektorlar bilan ifodalaymiz
- 2 Nega o'rganiladigan zich vektorlar?
- 3 CBOW arxitekturasini kontekstdan markazga**
- 4 Neyron tarmoqni o'qitish: yo'qotish va orqaga tarqatish
- 5 O'rganilgan embeddinglarni baholash
- 6 Xulosa va 6-amaliyotga ko'rsatmalar

CBOW intuitsiyasi: kontekstdan markaziy so'zni topamiz

Atrofdagi so'zlar beriladi, model esa markaziy so'zni bashorat qiladi.

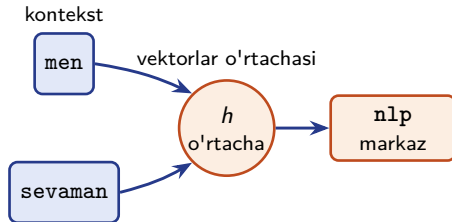
[men, sevaman] $\longrightarrow$?

Javob: nlp bo'lishi kerak. Buning uchun kontekst so'zlarining vektorlari o'rtachalanadi:

$$h = \frac{e_{\text{men}} + e_{\text{sevaman}}}{2}$$

So'ng shu h vektor orqali markaziy so'z ehtimoli hisoblanadi.

Bag of Words — tartib hisobga olinmaydi: [men, sevaman] va [sevaman, men] bir xil o'rtacha beradi.



Intuitsiya: kontekstning o'rtacha vektori markaziy so'zga ishora qilishi kerak.

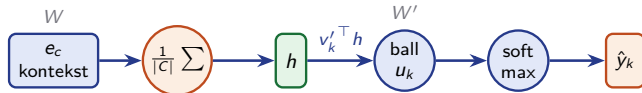
CBOW arxitekturasi: rasmiy ta'rif

$$h = \frac{1}{|C|} \sum_{c \in C} e_c, \quad u_k = v'_k{}^\top h, \quad \hat{y}_k = \frac{\exp(u_k)}{\sum_{k'} \exp(u_{k'})}.$$

Belgilashlar

- C — kontekst oynasi;
- e_c — kirish embeddingi;
- h — o'rtacha kontekst vektori;
- v'_k — chiqish vektori;
- u_k — k -so'z balli;
- $\hat{y}_k$ — softmax ehtimoli.

CBOWda ikki matritsa bor: kirish W va chiqish W' . Embedding sifatida ko'pincha W ishlatiladi.



Softmax ballarni ehtimollikka aylantiradi: exp qiymatlarni musbat qiladi, maxraj esa yig'indini 1 ga normallaydi.

Intuitsiya: W kontekst so'zlarini vektorga, W' esa h ni barcha so'zlar bo'yicha ballarga aylantiradi.

Keltirib chiqarish: nega kontekstni o'rtachalaymiz?

1-qadam. CBOWda kontekst so'zlari tartibsiz to'plam sifatida olinadi: qaysi so'z avval kelgani emas, qaysi so'zlar borligi muhim.

2-qadam. Har bir kontekst so'zi teng hissa qo'shadi. Shuning uchun ularning vektorlari o'rtachalanadi:

$$h = \frac{1}{|C|} \sum_{c \in C} e_c$$

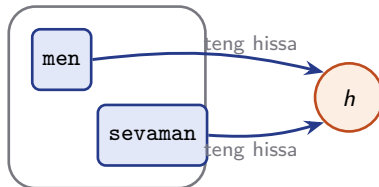
$|C| = 2m$ agar oyna radiusi m bo'lsa.

3-qadam. Bu amal sodda va tez:

$$h \text{ hisobi} = O(|C| \cdot d)$$

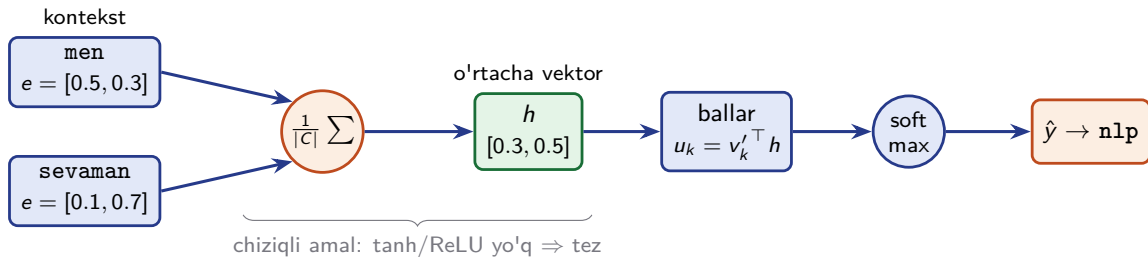
CBOWning yashirin qatlami murakkab neyron qatlam emas: u kontekst embeddinglarining oddiy o'rtachasidir.

bag: tartib hisobga olinmaydi



Intuitsiya: tartib yo'q $\Rightarrow$ simmetrik birlashtirish.
Eng sodda simmetrik birlashtirish — o'rtacha.

CBOW oqimi: kontekst $\rightarrow$ o'rtacha $\rightarrow$ bashorat



Intuitsiya: CBOW proyeksiya qatlami murakkab yashirin qatlam emas; u kontekst embeddinglarining oddiy o'rtachasidir.

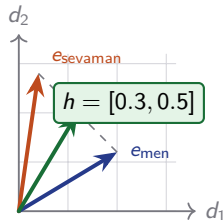
Qo'lda misol: CBOW proyeksiyasi $h = [0.3, 0.5]$

$$e_{\text{men}} = [0.5, 0.3], \quad e_{\text{sevaman}} = [0.1, 0.7].$$

Bu soddalashtirilgan o'quv misoli. Oyna: $2m = 2$ ta kontekst so'z.

Proyeksiya: kontekst vektorlarini o'rtachalaymiz

$$\begin{aligned} h &= \frac{1}{2} (e_{\text{men}} + e_{\text{sevaman}}) \\ &= \frac{1}{2} ([0.5, 0.3] + [0.1, 0.7]) \\ &= \frac{1}{2} [0.6, 1.0] \\ &= [0.3, 0.5]. \end{aligned}$$



Intuitsiya: ikki kontekst vektori berilganda h ularning **o'rtacha nuqtasi**, ya'ni kontekstning umumiy vakili bo'ladi.

$h = [0.3, 0.5]$ — markaziy so'zni bashorat qilish uchun ishlatiladigan **kontekst xulosasi**.

Sizning vazifangiz: boshqa kontekst uchun proyeksiya

Vazifa

Kontekst:

$$C = [\text{men}, \text{kitobni}]$$

$$e_{\text{men}} = [0.4, 0.2], \quad e_{\text{kitobni}} = [0.2, 0.6].$$

$$h = \frac{1}{|C|} \sum_{c \in C} e_c = ?$$

Sizning vazifangiz: boshqa kontekst uchun proyeksiya

Vazifa

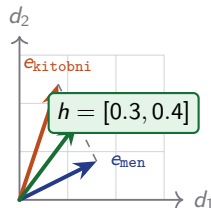
$$e_{\text{men}} = [0.4, 0.2], \quad e_{\text{kitobni}} = [0.2, 0.6], \quad h = ?$$

Javob

$$h = \frac{1}{2} ([0.4, 0.2] + [0.2, 0.6])$$

$$h = \frac{1}{2} [0.6, 0.8] = [0.3, 0.4].$$

Proyeksiya har bir koordinata bo'yicha o'rtacha olinadi.



Kod va formula: CBOW proyeksiyasi

Python kodi

```
import numpy as np

e_men = np.array([0.5, 0.3])
e_sevaman = np.array([0.1, 0.7])

# kontekst embeddinglari: |C| x d
context = np.stack([e_men, e_sevaman])

# h = (1 / |C|) * sum(e_c)
h = context.mean(axis=0)

print(h)    # [0.3 0.5]
```

Kod va formula mosligi

Kod	Formula
np.stack	$\{e_c\}_{c \in C}$
mean(axis=0)	$\frac{1}{ C } \sum_{c \in C} e_c$
h	h

$$h = [0.3, 0.5]$$

axis=0 har bir koordinata bo'yicha o'rtacha oladi.

CBOW tuzog'i: "bag" tartibni unutadi

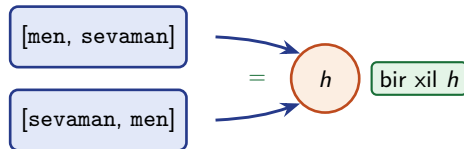
Tartib yo'qoladi

O'rtacha kommutativ:

$$h([\text{men}, \text{sevaman}]) = h([\text{sevaman}, \text{men}]).$$

Masalan, katta uy va uy katta — so'zlar bir xil, lekin tuzilma boshqa. CBOW esa tartibni hisobga olmaydi.

Bu so'z vektorlarini o'rganishda ko'pincha muammo emas, chunki maqsad gap sintaksisi emas, so'z ma'nosini o'rganishdir. Tartib muhim bo'lsa, pozitsion kodlash, RNN yoki Transformer kerak bo'ladi.



Intuitsiya: ikki xil tartib, lekin bitta o'rtacha. Demak, tartib ma'lumoti yo'qoladi.

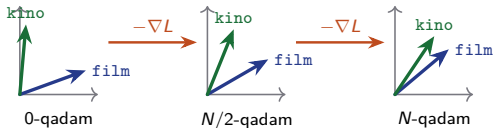
Prezentatsiya rejasi

- 1 Kirish: so'zlarni o'rganilgan vektorlar bilan ifodalaymiz
- 2 Nega o'rganiladigan zich vektorlar?
- 3 CBOW arxitekturasini kontekstdan markazga
- 4 Neyron tarmoqni o'qitish: yo'qotish va orqaga tarqatish
- 5 O'rganilgan embeddinglarni baholash
- 6 Xulosa va 6-amaliyotga ko'rsatmalar

Intuitsiya: xatoni kamaytirish uchun vektorlarni yangilaymiz

O'qitish sikli har bir kontekst–markaz jufti uchun takrorlanadi:

- 1 **Oldinga hisoblash** (*forward*): kontekstdan h olinadi va softmax orqali bashorat $\hat{y}$ hisoblanadi.
- 2 **Yo'qotish** (*loss*): model bashorati to'g'ri markaz so'zdan qanchalik uzoqligi o'lchanadi.
- 3 **Orqaga tarqatish** (*backward*): W va W' xatoni kamaytiradigan yo'nalishda yangilanadi.



O'rganish qoidasi:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Bu yerda θ — yangilanadigan parametrlar, masalan W va W' ;
 η — o'rganish tezligi.

Intuitsiya: har bir qadam vektorlarni ozgina suradi. Minglab misollardan keyin film va kino kabi kontekstdosh so'zlar orasidagi burchak kichrayishi mumkin.

Muhim fikr: o'xshash kontekstlarda ko'p uchraydigan so'zlar ko'p yangilanishlardan keyin vektor fazosida yaqinlashishga moyil bo'ladi.

Yo'qotish: cross-entropy nimani jazolaydi?

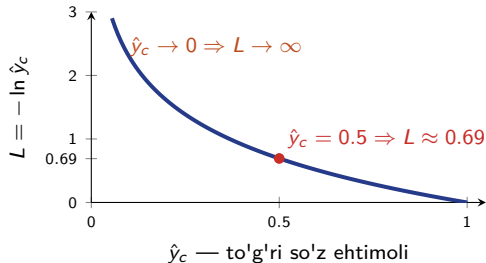
$$L = - \sum_k y_k \ln \hat{y}_k$$

y one-hot bo'lsa: $L = - \ln \hat{y}_c$

Ma'nosi

- c — haqiqiy markaz so'z indeksi;
- $\hat{y}_c$ — model to'g'ri so'zga bergan ehtimol;
- L — model xatosining sonli o'lchovi.

Agar model to'g'ri so'zga yuqori ehtimol bersa, L kichik. Agar past ehtimol bersa, L katta bo'ladi.



Cross-entropy modelning “to'g'ri javobga ishonchi”ni baholaydi. Ishonch yuqori bo'lsa — jazo kichik.

Gradient: bashorat minus haqiqat

Softmax + cross-entropy uchun:

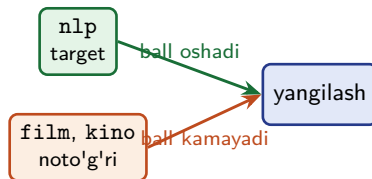
$$\frac{\partial L}{\partial u_k} = \hat{y}_k - y_k$$

Bu nimani bildiradi?

- u_k — softmaxdan oldingi ball;
- $\hat{y}_k$ — model bergan ehtimol;
- y_k — haqiqiy javob: targetda 1, boshqalarda 0.

Gradient — **model bashorati** va **haqiqiy javob** orasidagi farq. Yangilash shu farqni kamaytirishga harakat qiladi.

So'z	$\hat{y}_k$	y_k	$\hat{y}_k - y_k$
nlp	0.60	1	-0.40
film	0.25	0	0.25
kino	0.15	0	0.15



Gradient descentda manfiy gradient target so'z ballini oshirishga, musbat gradient esa noto'g'ri so'zlar ballini kamaytirishga olib boradi.

Keltirib chiqarish: $\partial L / \partial u_k = \hat{y}_k - y_k$

1-qadam. Lossni softmax orqali ochamiz:

$$L = -\ln \hat{y}_c = -u_c + \ln \sum_j e^{u_j}$$

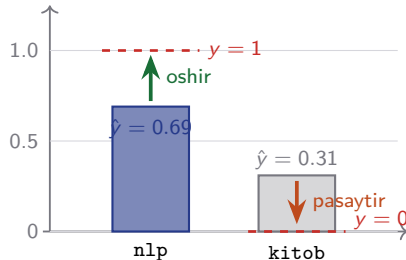
2-qadam. u_k bo'yicha hosila olamiz:

$$\frac{\partial L}{\partial u_k} = -\frac{\partial u_c}{\partial u_k} + \frac{e^{u_k}}{\sum_j e^{u_j}}$$
$$\frac{\partial u_c}{\partial u_k} = y_k, \quad \frac{e^{u_k}}{\sum_j e^{u_j}} = \hat{y}_k$$

3-qadam. Natija:

$$\frac{\partial L}{\partial u_k} = \hat{y}_k - y_k$$

Target so'zda $y_k = 1$, boshqalarda $y_k = 0$. Shuning uchun gradient — **bashorat minus haqiqat**.



Gradient descentda target so'z balli oshadi, noto'g'ri so'zlar balli pasayadi.

Qo'lda misol: forward, yo'qotish va gradient

Berilganlar:

$$h = [0.3, 0.5], \quad v'_{\text{nlp}} = [1, 1], \quad v'_{\text{kitob}} = [0, 0].$$

1. Ballar:

$$u_{\text{nlp}} = [1, 1] \cdot [0.3, 0.5] = 0.8$$

$$u_{\text{kitob}} = [0, 0] \cdot [0.3, 0.5] = 0.$$

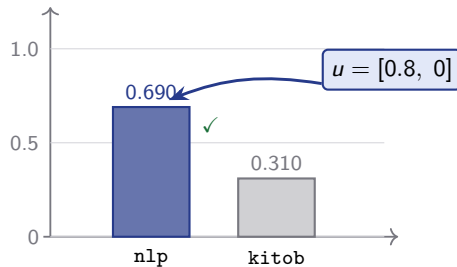
2. Softmax:

$$\hat{y}_{\text{nlp}} = \frac{e^{0.8}}{e^{0.8} + e^0} \approx 0.690, \quad \hat{y}_{\text{kitob}} \approx 0.310.$$

3. Loss va gradient:

$$L = -\ln(0.690) \approx 0.371$$

$$\frac{\partial L}{\partial u_{\text{nlp}}} = 0.690 - 1 = -0.310, \quad \frac{\partial L}{\partial u_{\text{kitob}}} = 0.310 - 0 = 0.310.$$



Intuitsiya: target so'z nlp uchun gradient manfiy, shuning uchun gradient descent uning ballini oshiradi. kitob uchun gradient musbat — ball pasayadi.

Sizning vazifangiz: teng ballarda yo'qotish

Vazifa

Ikkala ball ham teng:

$$u_{\text{nlp}} = 0, \quad u_{\text{kitob}} = 0.$$

Target so'z — nlp.

$$\hat{y}_{\text{nlp}} = ?, \quad L = ?$$

Sizning vazifangiz: teng ballarda yo'qotish

Vazifa

$$u_{\text{nlp}} = 0, \quad u_{\text{kitob}} = 0, \quad \text{target} = \text{nlp}.$$

$$\hat{y}_{\text{nlp}} = ?, \quad L = ?$$

Javob

Softmax:

$$\hat{y}_{\text{nlp}} = \frac{e^0}{e^0 + e^0} = \frac{1}{2} = 0.5.$$

Loss:

$$L = -\ln \hat{y}_{\text{nlp}} = -\ln 0.5 = \ln 2 \approx 0.693.$$

Bu — model hali ajrata olmayotgan holat: ikki nomzodga teng ehtimol beradi.



Intuitsiya: teng ball $\Rightarrow$ 50/50 taxmin. O'qitishdan keyin target so'z balli yuqoriroq bo'lishi kerak.

Kod va formula: softmax, yo'qotish, gradient

Python kodi

```
import numpy as np

h = np.array([0.3, 0.5])
Wout = np.array([[1, 1],          # v'_nlp
                  [0, 0]])        # v'_kitob

u = Wout @ h                      # [0.8, 0.0]

z = u - u.max()                  # barqaror softmax
yhat = np.exp(z) / np.exp(z).sum()

y = np.array([1, 0])            # target: nlp
loss = -np.log(yhat[0])
grad_u = yhat - y
```

Kod va formula mosligi

Kod	Formula
$u = W_{out} @ h$	$u_k = v'_k{}^T h$
$\text{softmax}(u)$	$\hat{y}_k = \frac{e^{u_k}}{\sum_j e^{u_j}}$
loss	$L = -\ln \hat{y}_c$
grad_u	$\nabla_u L = \hat{y} - y$

$$u = [0.8, 0.0]$$

$$\hat{y} = [0.690, 0.310]$$

$$L = 0.371, \quad \nabla_u L = [-0.310, 0.310].$$

Katta lug'atda to'liq softmax qimmat;
Word2Vecda ko'pincha **manfiy namunalash**
ishlatiladi.

O'qitish tuzog'i: to'liq softmax qimmat

Muammo

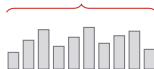
$$\hat{y}_k = \frac{e^{u_k}}{\sum_{j \in V} e^{u_j}}$$

Maxraj har qadamda **barcha lug'at** bo'yicha hisoblanadi. Agar $|V|$ katta bo'lsa (10^5+), bu **juda sekin**.

Amaliy yechim

To'liq softmax o'rniga: **salbiy namunalash** (*negative sampling*) yoki **ierarxik softmax** (*hierarchical softmax*) ishlatiladi.

to'liq softmax: $|V|$ ta



negative sampling: $1 + K$ ta



To'liq softmax **hamma** so'zni tekshiradi; negative sampling faqat **target + bir nechta salbiy namuna** bilan ishlaydi.

Natija: hisoblash **ancha tezlashadi**.

Prezentatsiya rejasi

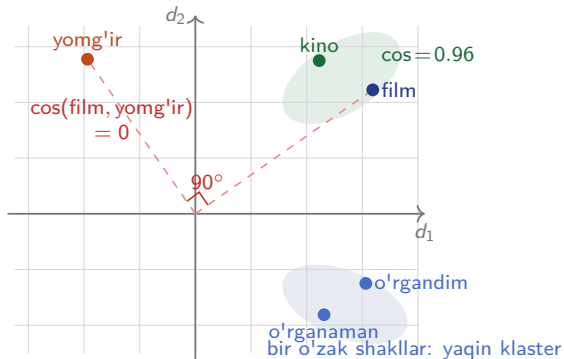
- 1 Kirish: so'zlarni o'rganilgan vektorlar bilan ifodalaymiz
- 2 Nega o'rganiladigan zich vektorlar?
- 3 CBOW arxitekturasini kontekstdan markazga
- 4 Neyron tarmoqni o'qitish: yo'qotish va orqaga tarqatish
- 5 O'rganilgan embeddinglarni baholash**
- 6 Xulosa va 6-amaliyotga ko'rsatmalar

Intuitsiya: yaxshi embedding fazosida yaqin ma'no — yaqin nuqta

O'qitishdan keyin embedding fazosini **tekshiramiz**: ma'noda yaqin so'zlar yaqin joylashganmi?

Ikki vosita: **kosinus o'xshashligi** (ikki so'z qanchalik yaqin) va **analogiya** (munosabatlar saqlanadimi).

Intuitsiya: korpusdan o'rganilgan zich fazoda **kontekstdosh so'zlar yaqin**: $\text{film} \approx \text{kino}$; yomg'ir uzoq. Baholash shu xususiyatni o'lchaydi.



Baholash o'lchovlari: kosinus o'xshashligi va analogiya

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

analogiya: $v(\text{Toshkent}) - v(\text{O'zbekiston}) + v(\text{Rossiya}) \approx v(\text{Moskva})$

Eslatma: analogiya aniq tenglik emas ($\approx$ eng yaqin qo'shni); bu xossa faqat **katta** korpusda o'rganilgan vektorlarda paydo bo'ladi.

Kosinus o'xshashligi ikki vektor orasidagi *burchakni* o'lchaydi: $+1$ (bir xil yo'nalish) dan -1 (qarama-qarshi) gacha.

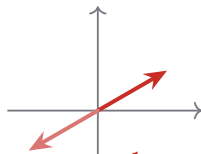
Analogiya	munosabatlar	yo'nalish
sifatida	saqlanishini	tekshiradi:
"O'zbekiston→Toshkent"		siljishi
"Rossiya→Moskva"	bilan bir xil (poytaxt munosabati).	



$\cos \approx 1$
yaqin ma'no



$\cos = 0$
bog'liqsiz



$\cos = -1$
qarama-qarshi

Intuitsiya: uzunlik emas, **burchak** muhim — shuning uchun kosinus.

Keltirib chiqarish: nega kosinus, xom skalyar ko'paytma emas?

1-qadam. Skalyar ko'paytma yo'nalish va uzunlikni aralashtiradi:

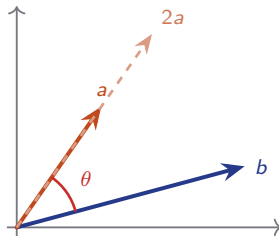
$$a \cdot b = \|a\| \|b\| \cos \theta$$

2-qadam. So'z chastotasi vektor uzunligiga ta'sir qiladi — ko'p uchragan so'z uzunroq vektor olishi mumkin. Xom ko'paytma buni adolatsiz qiladi.

3-qadam. Uzunliklarga bo'lib, **faqat burchakni** qoldiramiz:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|} = \cos \theta$$

Vektorni $2\times$ uzaytirsak ham $\cos$ o'zgarmaydi — ma'no yaqinligi chastotaga bog'liq bo'lmaydi.



$\cos(a, b) = \cos(2a, b)$
uzunlik o'zgaradi — **cos o'zgarmaydi**

Intuitsiya: uzunlikni bo'lib tashlaymiz, **burchak** qoladi.

Qo'lda misol: $\cos(\text{film}, \text{kino}) = 0.96$

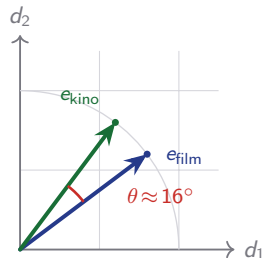
O'rganilgan zich vektorlar ($d = 2$, soddalashtirilgan o'quv misoli): $e_{\text{film}} = [0.8, 0.6]$, $e_{\text{kino}} = [0.6, 0.8]$.

$$a \cdot b = 0.8 \cdot 0.6 + 0.6 \cdot 0.8 = 0.96$$

$$\|a\| = \|b\| = \sqrt{0.8^2 + 0.6^2} = 1$$

$$\cos(\text{film}, \text{kino}) = \frac{0.96}{1 \cdot 1} = 0.96$$

One-hot da bu qiymat 0 edi. O'rganilgan zich fazoda 0.96 — model film va kino ni yaqin deb biladi.



Intuitsiya: yaqin ma'no $\Rightarrow$ deyarli bir yo'nalish.

Sizning vazifangiz: $\cos(\text{film}, \text{yomg'ir}) = ?$

Vazifa

$e_{\text{film}} = [0.8, 0.6]$, $e_{\text{yomg'ir}} = [-0.6, 0.8]$ (ikkalasi uzunligi 1).

$\cos(\text{film}, \text{yomg'ir}) = ?$

Bu nimani bildiradi?

Sizning vazifangiz: $\cos(\text{film}, \text{yomg'ir}) = ?$

Vazifa

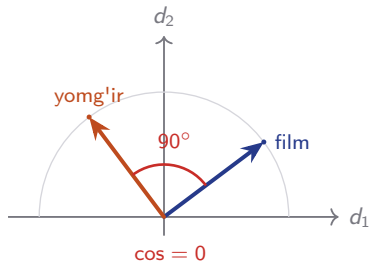
$$e_{\text{film}} = [0.8, 0.6], e_{\text{yomg'ir}} = [-0.6, 0.8]. \quad \cos = ?$$

Javob

$$a \cdot b = 0.8 \cdot (-0.6) + 0.6 \cdot 0.8 = -0.48 + 0.48 = 0$$

$$\cos(\text{film}, \text{yomg'ir}) = \frac{0}{1 \cdot 1} = 0$$

Vektorlar **ortogonal** (90°): model bu so'zlarni bog'liqsiz deb biladi — film va kino dan farqli o'laroq.



Intuitsiya: $90^\circ \Rightarrow \cos = 0$ — ma'no bog'liqsiz.

Kod va formula: kosinus o'xshashligi va most_similar

Python kodi

```
import numpy as np
def cosine(a, b):
    return a @ b / (np.linalg.norm(a)
                    * np.linalg.norm(b))

e_film = np.array([0.8, 0.6])
e_kino = np.array([0.6, 0.8])
print(round(cosine(e_film, e_kino), 2))    # 0.96

# eng o'xshash so'zlar: butun lug'at bo'yicha
def most_similar(e_q, W, V, top=3):
    sims = [cosine(e_q, W[i])
            for i in range(len(V))]
    order = np.argsort(sims)[::-1]
    return [(V[i], round(sims[i], 2))
            for i in order[:top]]
```

Kod va formula mosligi

Kod	Formula
$a @ b$	$a \cdot b$
$\text{norm}(a)$	$\ a\ $
$\text{cosine}(a, b)$	$\frac{a \cdot b}{\ a\ \ b\ }$
most_similar	eng katta cos

Gensim — Python kutubxonasi. Unda tayyor funksiya bor: `model.wv.most_similar("film")`. U film ga eng yaqin so'zlarni kosinus o'xshashligi bo'yicha qaytaradi.

Baholash tuzog'i: kam yoki shovqinli ma'lumot — sifatsiz embedding

Embedding korpus sifatiga bog'liq

Kichik yoki shovqinli korpusda kontekstlar yetarli emas — vektorlar tasodifga yaqin qoladi. $\cos(\text{film}, \text{kino})$ baland chiqmaydi.

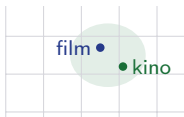
Amaliy maslahat

Ko'proq va tozalangan matn; kam uchraydigan so'zlar uchun **subword** (*so'z osti bo'laklari*; fastText) yoki minimal chastota chegarasi. O'zbek uchun normallashtirish (lotin/kirill, qo'shimchalar) muhim.

kam ma'lumot: tarqoq



ko'p ma'lumot: klaster



Intuitsiya: yaxshi embedding **ko'p kontekstdan** tug'iladi.

Prezentatsiya rejasi

- 1 Kirish: so'zlarni o'rganilgan vektorlar bilan ifodalaymiz
- 2 Nega o'rganiladigan zich vektorlar?
- 3 CBOW arxitekturasini kontekstdan markazga
- 4 Neyron tarmoqni o'qitish: yo'qotish va orqaga tarqatish
- 5 O'rganilgan embeddinglarni baholash
- 6 Xulosa va 6-amaliyotga ko'rsatmalar

O'zbek tili uchun ahamiyati: agglutinatsiya va lug'at portlashi

Muammo (5-ma'ruza takrori)

Agglutinativ tilda bir o'zak ko'p shakl beradi: o'rganaman, o'rgandim, o'rganadi... One-hot da har biri **alohida token** — lug'at portlaydi, o'xshashlik yo'q.

Embedding nima beradi

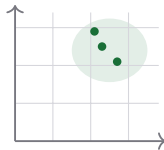
Kontekstdan o'rganilgan zich vektorlar bir o'zak shakllarni **yaqin klasterga** joylaydi. Subword (fastText) buni yanada kuchaytiradi.

siyrak (one-hot)

o'rganaman	1	0	0
o'rgandim	0	1	0
o'rganadi	0	0	1

3 alohida o'q $\Rightarrow \cos = 0$

zich (embedding)



bir o'zak $\Rightarrow$ yaqin klaster

Sintez: siyrak (one-hot) va zich (embedding) taqqoslash

Xususiyat	One-hot (siyrak)	Embedding (zich)
O'lcham	$ V \sim 10^5$	$d \sim 100-300$
To'ldirilishi	siyrak (bitta 1)	zich (haqiqiy sonlar)
Manba	qoidaviy (indeks)	korpusdan o'rganilgan
cos(film, kino)	0	0.96
Ma'no o'xshashligi	yo'lq	bor
O'zbek shakllari	har biri alohida	yaqin klaster

Asosiy g'oya

Ma'noni kontekstdan o'rganamiz. CBOW kontekstni o'rtachalab markaziy so'zni bashorat qiladi; natijada yaqin ma'noli so'zlar yaqin vektor oladi ($\cos = 0.96$).

Seminal manba va munozara: Mikolov va boshqalar (2013)

Word2Vec (CBOW va Skip-gram) Mikolov va boshqalar tomonidan 2013-yilda taklif qilingan. G'oya: so'z ma'nosini katta korpusdan, kontekstni bashorat qilish orqali o'rganish. Salbiy namunalash bilan juda samarali o'qiladi.

Munozara va cheklovlar

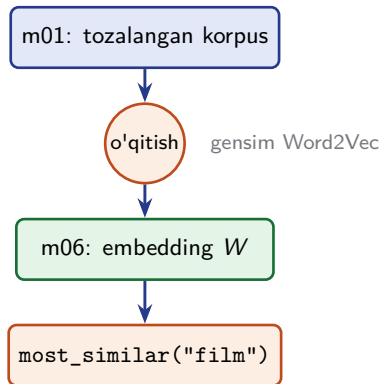
CBOW tez va kam uchraydigan so'zlarni silliqlaydi; Skip-gram kam ma'lumotda yaxshiroq. Ikkalasi ham **bitta so'z = bitta vektor** (polisemiyaning — ko'p ma'nolilikni — ajratmaydi) va morfologiyani to'g'ridan-to'g'ri ko'rmaydi — subword (fastText) va kontekstual modellar (7-ma'ruza) shu cheklovlarni hal qiladi.

Bugungi maqsadlar bajarildi

- 1 ✓ **Keltirib chiqardik:** one-hot ortogonal $\Rightarrow \cos = 0$ — siyrak ifoda o'xshashlikni ko'rsata olmaydi.
- 2 ✓ **Tushuntirdik:** CBOW kontekst vektorlarini o'rtachalaydi — $h = [0.3, 0.5]$.
- 3 ✓ **Hisobladik:** softmax, yo'qotish va gradient bir qadami — $\hat{y}(\text{nlp}) = 0.690$, $L = 0.371$, $\hat{y} - y = -0.310$.
- 4 ✓ **Baholadik:** o'rganilgan embeddinglarni kosinus orqali — $\cos(\text{film}, \text{kino}) = 0.96$.

Bitta jumla bilan: **ma'noni kontekstdan o'rgandik** va uni vektor fazoda o'lchadik.

Ertaga 6-amaliyot: Word2Vec ni o'zbek korpusida o'rgatish



6-amaliyotda qurasiz

```
from gensim.models import Word2Vec
# jumlarlar: tokenlangan o'zbek korpusi (m01)
model = Word2Vec(sentences, vector_size=100,
                  window=1, sg=0, # CBOW
                  negative=5, min_count=5)
# o'rganilgan embeddinglar (m06)
print(model.wv.most_similar("film", topn=3))
# tekshiruv: film va kino yaqinmi?
assert model.wv.similarity(
    "film", "kino") > 0.5
```

sg=0 — CBOW; negative=5 — salbiy namunalash;
window=1 — oyna radiusi $m = 1$, bugungi $|C| = 2m = 2$ ga
mos. Natija tasodifiy boshlanishga bog'liq — shuning uchun
assert chegarasi yumshoq (> 0.5).

7-ma'ruzaga ko'prik: so'z vektori bor, lekin tartib yo'q

Word2Vec har bir so'zga bitta **statik vektor** beradi. Lekin gap — bu **so'zlar ketma-ketligi**.

Masalan: katta uy va uy katta.

So'zlar bir xil, lekin tartib o'zgarsa, gapning tuzilishi ham o'zgaradi. CBOW kontekstni o'rtachalaydi, ya'ni tartibni saqlamaydi.

RNN (*Recurrent Neural Network* — rekurent neyron tarmoq) so'zlarni **ketma-ket** o'qiydi va yashirin holat saqlaydi. Shuning uchun u tartibni hisobga oladi.

Ya'ni: statik so'z vektoridan → ketma-ketlik modeliga.

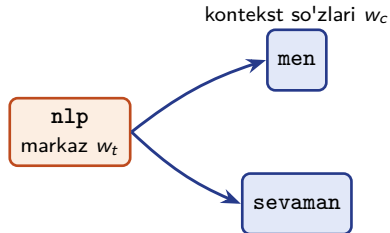
- ① Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781.
- ② Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. NeurIPS. (salbiy namunalash)
- ③ Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). *Enriching Word Vectors with Subword Information*. (fastText)
- ④ Řehůřek, R., & Sojka, P. *Gensim — Topic Modelling for Humans*.
- ⑤ Jurafsky, D., & Martin, J. H. *Speech and Language Processing*, 6-bob: *Vector Semantics and Embeddings*.

Ilova S1: Skip-gram — CBOW ning teskarisi

$$L_{SG} = - \sum_{c \in C} \ln P(w_c | w_t), \quad P(w_c | w_t) = \frac{\exp(v'_c{}^\top v_t)}{\sum_{k \in V} \exp(v'_k{}^\top v_t)}$$

CBOW **kontekstdan markazni** bashorat qiladi.
Skip-gram esa teskarisi: **markazdan kontekstni**
bashorat qiladi.

Markaziy so'z w_t berilganda, model atrofdagi har bir kontekst so'zi w_c ehtimolini oshirishga o'rganadi.



Skip-gram kam uchraydigan so'zlar uchun ko'pincha yaxshiroq ishlaydi, lekin CBOWga nisbatan sekinroq o'qishi mumkin. Gensimda: `sg=1`.

Intuitsiya: bitta markaz so'zdan bir nechta kontekst so'z bashorat qilinadi.

Ilova S2: Salbiy namunalash (*negative sampling*)

$$L_{\text{neg}} = -\ln \sigma(v_o'^{\top} h) - \sum_{i=1}^K \ln \sigma(-v_{n_i}'^{\top} h), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

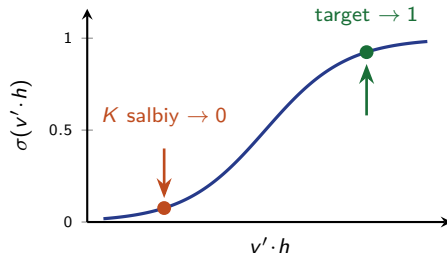
To'liq softmax o'rniga model faqat:

- **target** so'z o ballini oshiradi:

$$\sigma(v_o'^{\top} h) \rightarrow 1;$$

- K ta **salbiy namuna** n_i ballini pasaytiradi:

$$\sigma(v_{n_i}'^{\top} h) \rightarrow 0.$$



Maxraj butun lug'at bo'yicha hisoblanmaydi.
Hisoblash $|V|$ ta so'zdan $K + 1$ ta so'zga kamayadi.
Odatda $K = 5-20$.

Intuitsiya: target juftlik "haqiqiy" deb yuqoriga suriladi, salbiy namunalar esa "mos emas" deb pastga tushiriladi.